

算法设计与分析

第5章 回溯法



- 理解回溯法的深度优先策略
- 掌握用回溯法解题的算法框架
 - (1) 递归回溯 (2) 迭代回溯
 - (3) 子集树算法框架 (4) 排列树算法框架
- 通过应用范例学习回溯法的设计策略
 - (1) 装载问题 (2) 批处理作业调度 (3) 符号三角形问题 (4) n皇后问题
 - (5) 0-1背包问题 (6) 最大团问题 (7) 图的m着色问题 (8) 旅行售货员问题
 - (9) 圆排列问题 (10) 连续邮资问题

5.1 回溯法



- 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。
- 回溯法的基本做法是搜索，或是一种组织得井井有条的，能避免不必要搜索的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。
- 回溯法在问题的解空间树中，按深度优先策略，从根节点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该节点是否包含问题的解。如果肯定不包含，则跳过对该节点为根的子树的搜索，逐层向其祖先节点回溯；否则，进入该子树，继续按深度优先策略搜索。

5.1 回溯法



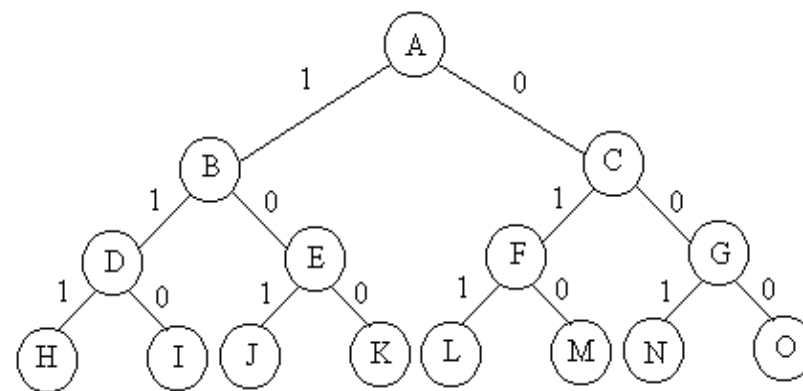
问题的解空间

- 问题的解向量：回溯法希望一个问题的解能够表示成一个 n 元式 $(x_1, x_2, \dots, x_n)$ 的形式。
- 显约束：对分量 x_i 的取值限定。
- 隐约束：为满足问题的解而对不同分量之间施加的约束。
- 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。

$$W=[16,15,15] \quad p=[45,25,25] \quad C=30$$

- $n=3$ 时的0-1背包问题用完全二叉树表示的解空间



生成问题状态的基本方法

- 扩展节点：一个正在产生儿子的节点称为扩展节点。
也就是说，扩展节点是搜索树中还没有完全探索的节点。
- 活节点：一个自身已生成但其儿子还没有全部生成的节点称做活节点。
也就是说，活节点是搜索树中还有未探索分支的节点。
- 死节点：一个所有儿子已经产生的节点称做死节点。
也就是说，死节点是搜索树中所有分支都已被探索或者不满足要求的节点。
- 深度优先的问题状态生成法：如果对一个扩展节点R，一旦产生了它的一个儿子C，就把C当做新的扩展节点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展节点，继续生成R的下一个儿子（如果存在）。
- 广度优先的问题状态生成法：在一个扩展节点变成死节点之前，它一直是扩展节点
- 回溯法：为了避免生成那些不可能产生最优解的问题状态，要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活节点，以减少问题的计算量。具有限界函数的深度优先生成法称为回溯法。

回溯法的基本思路

- 从根开始，以深度优先搜索的方式进行搜索。
- 根节点是活节点并且是当前的扩展节点。在搜索的过程中，当前的扩展节点向纵深方向移向一个新节点，判断该新节点是否满足隐约束。
- 如果满足，则新节点成为活节点，并且成为当前的扩展节点，继续深一层的搜索；
- 如果不满足，则换该新节点的兄弟节点（扩展节点的其他分支）继续搜索；
- 如果新节点没有兄弟节点，或其兄弟节点已全部搜索完毕，则扩展节点成为死节点，搜索回溯到其父节点处继续进行。
- 搜索过程直到找到问题的解或根节点变成死节点为止。

回溯法的基本思想

- (1) 针对所给问题，定义问题的解空间；
- (2) 确定易于搜索的解空间结构；
- (3) 以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

常用剪枝函数：

- 用 **约束函数** 在扩展节点处剪去不满足约束的子树；
 - 用 **限界函数** 剪去得不到最优解的子树。
-
- 用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根节点到当前扩展节点的路径。如果解空间树中从根节点到叶节点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。

回溯法的基本思想

- 明确搜索范围（定义问题的解空间）

解的形式：一个 n 元式 $(x_1, x_2, \dots, x_n)$

确定 x_i 的取值范围（显约束），确定解空间的大小。

- 确定解空间的组织结构——树或图

- 搜索解空间（隐约束）

确定是否能够导致可行解——约束条件

确定是否能够导致最优解——限界条件

- 用约束函数在扩展节点处剪去不满足约束的子树即：导致不可行解的节点；
- 用限界函数剪去得不到最优解的子树。即：导致非最优解的节点。

子集树与排列树

- 当所给问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，解空间为子集树。
(装载问题、符号三角形问题、0-1背包问题、最大团问题)
- 当所给问题是从 n 个元素的集合 S 中找出满足某种性质的排列时，解空间为排列树。
(批处理作业调度、 n 后问题、旅行售货员问题、圆排列问题、电路板排列问题)

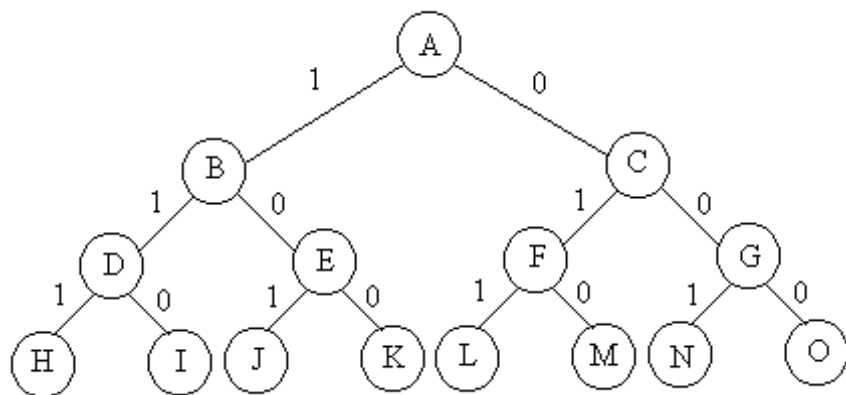
子集树

- 当所给的问题是从 n 个元素组成的集合 S 中找出满足某种性质的一个子集时，相应的解空间树称为子集树。
- 此类问题解的形式为 n 元组 $(x_1, x_2, \dots, x_n)$ ，分量 $x_i (i=1, 2, \dots, n)$ 表示第 i 个元素是否在要找的子集中。
- x_i 的取值为0或1， $x_i=0$ 表示第 i 个元素不在要找的子集中； $x_i=1$ 表示第 i 个元素在要找的子集中。

排列树

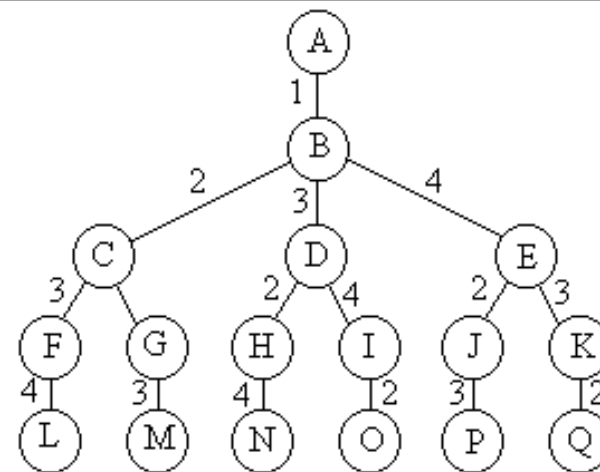
- 当所给的问题是从 n 个元素的排列中找出满足某种性质的一个排列时，相应的解空间树称为排列树。
- 此类问题解的形式为 n 元组 $(x_1, x_2, \dots, x_n)$ ，分量 $x_i (i=1, 2, \dots, n)$ 表示第 i 个位置的元素是 x_i 。 n 个元素组成的集合为 $S=\{1, 2, \dots, n\}$ ， $x_i \in S - \{x_1, x_2, \dots, x_{i-1}\}$ ， $i=1, 2, \dots, n$ 。

子集树与排列树



- 遍历子集树需 $\Omega(2^n)$ 计算时间

```
void backtrack (int t){  
    if (t>n) output(x); // 到达叶子节点  
    else  
        for (int i=0;i<=1;i++) { // 两个分支  
            x[t]=i;  
            if (legal(t)) backtrack(t+1);  
        }  
}
```



- 遍历排列树需要 $\Omega(n!)$ 计算时间

```
void backtrack (int t){  
    if (t>n) output(x);  
    else  
        for (int i=t;i<=n;i++) {  
            // 完成全排列  
            swap(x[t], x[i]);  
            if (legal(t)) backtrack(t+1);  
            swap(x[t], x[i]);  
        }  
}
```

5.6 0-1背包问题

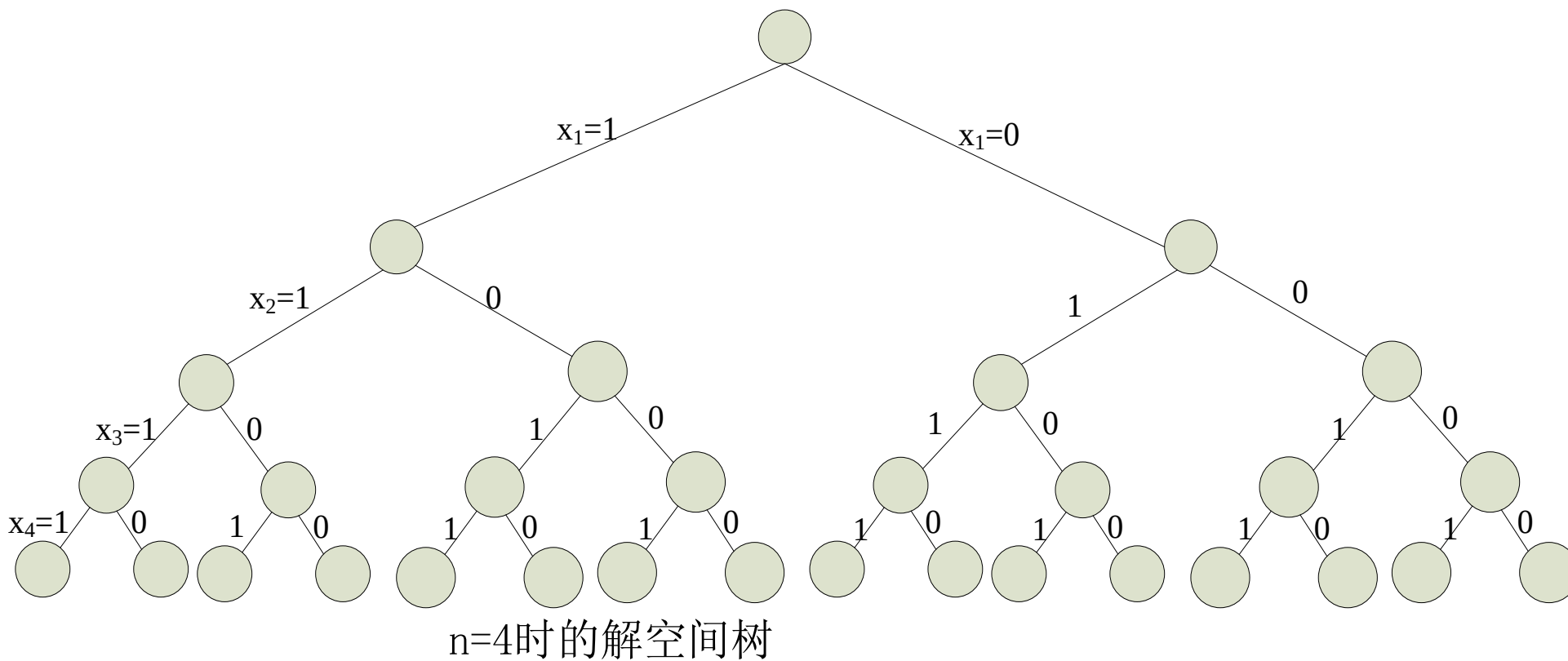


- 问题描述：给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 W 。一个物品要么全部装入背包，要么全部不装入背包，不允许部分装入。装入背包的物品的总重量不超过背包的容量。问应如何选择装入背包的物品，使得装入背包中的物品总价值最大？
- 定义问题的解空间
解的形式 $(x_1, x_2, \dots, x_n)$ ，其中 $x_i=0$ 或 1
 $x_i=0$: 第 i 个物品不装入背包
 $x_i=1$: 第 i 个物品装入背包

5.6 0-1背包问题



确定解空间



5.6 0-1背包问题



搜索解空间

- 约束条件

- $w_i \leq c'$ (c' 为背包的剩余容量) 也就是: $\sum_{i=1}^n w_i x_i \leq W$

- 限界条件: $cp+rp>bestp$ 其中

- cp : 当前已装入背包的物品的总价值
- rp : 剩余不知道是否装入的物品的总价值
- $bestp$: 当前已经找到的最优解的价值

- 搜索过程

- 以 $n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$ 为例展示搜索过程

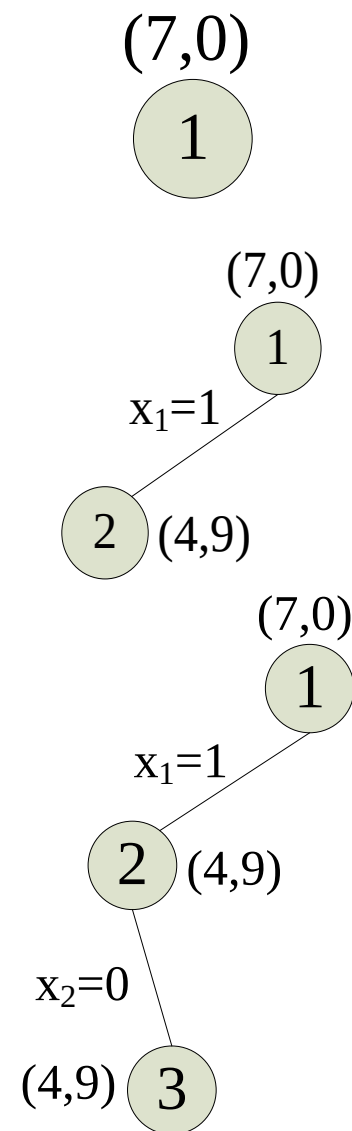
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 根节点是活节点并且是当前的扩展节点
- 第一个物品的重量为3, $3 < 7$, 满足约束条件
- 第二个物品的重量为5, $5 > (7-3)$, 不满足约束条件
- $cp=9$, $rp=11$, $bestp=0$, $cp+rp > bestp$, 满足限界条件。



5.6 0-1背包问题

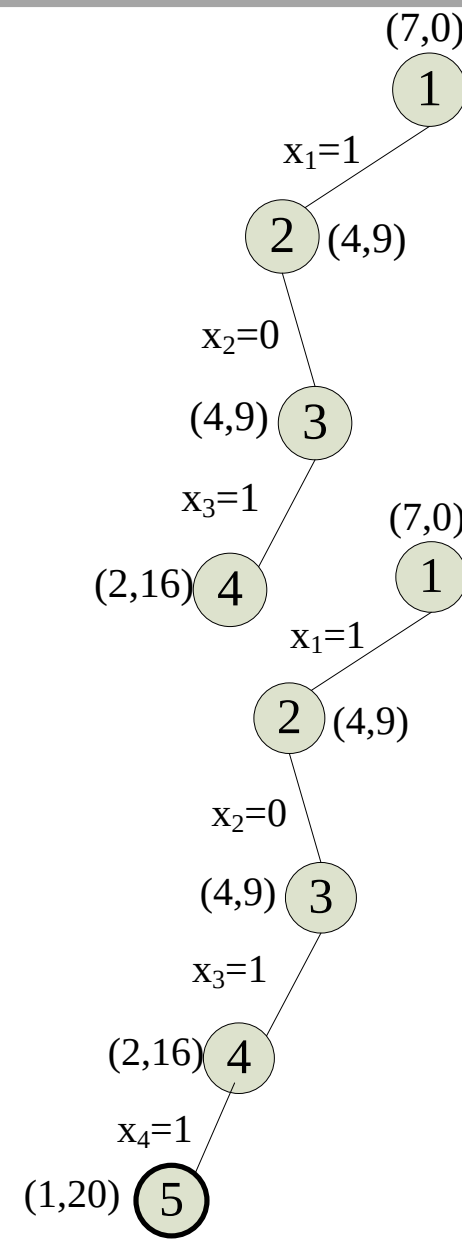


搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 第3个物品的重量是2，背包的剩余容量为4，满足约束条件
- 第4个物品的重量为1，背包的剩余容量为2，满足约束条件。

现在已经到达叶子，找到当前最优解， $bestp=9+7+4=20$ 。



5.6 0-1背包问题

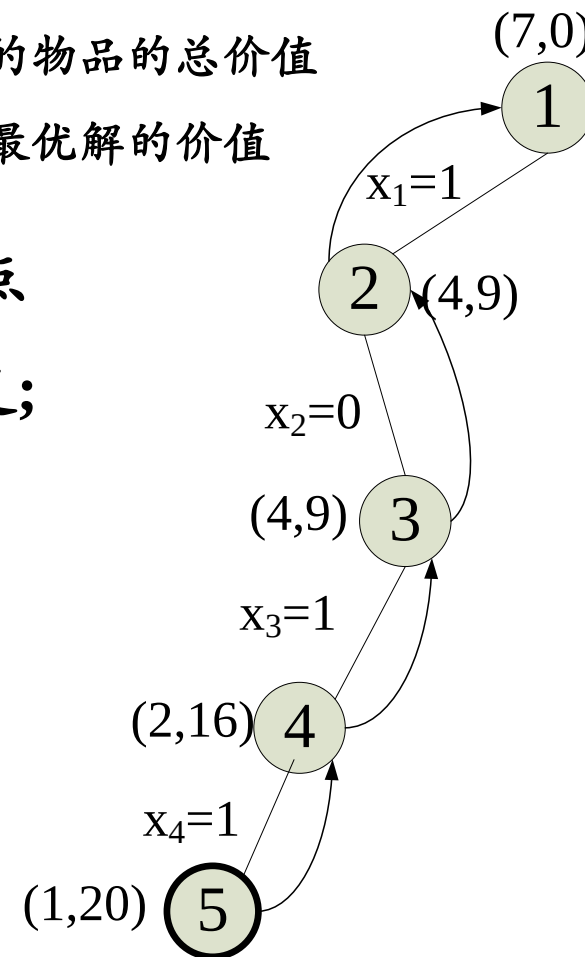


搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 此时要回溯到离节点5最近的活节点4, 节点4再次成为扩展节点
- 限界条件 $cp=16$, $rp=0$, $bestp=20$ $cp+rp < bestp$, 限界条件不满足;
- 回溯到最近的活节点3, 节点3再次成为扩展节点
- 限界条件是否满足, $cp=9$, $rp=4$ (右子树的可能值 $x_3=0, x_4=1$),
 $bestp=20$, $cp+rp < bestp$, 限界条件不满足
- 回溯到最近的活节点2, 继续回溯到节点1

- cp : 当前已装入背包的物品的总价值
- rp : 剩余不知道是否装入的物品的总价值
- $bestp$: 当前已经找到的最优解的价值



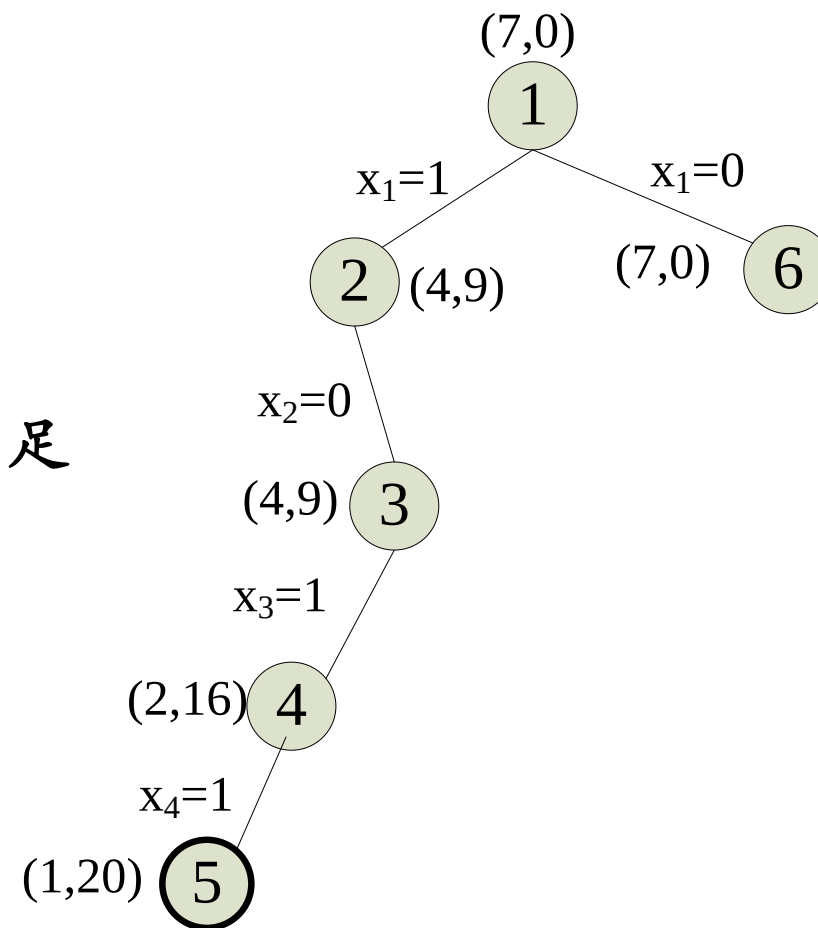
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 扩展节点1沿着右分支继续扩展, $cp=0$, $rp=21$
($10+7+4$) , $bestp=20$, $cp+rp>bestp$, 限界条件满足



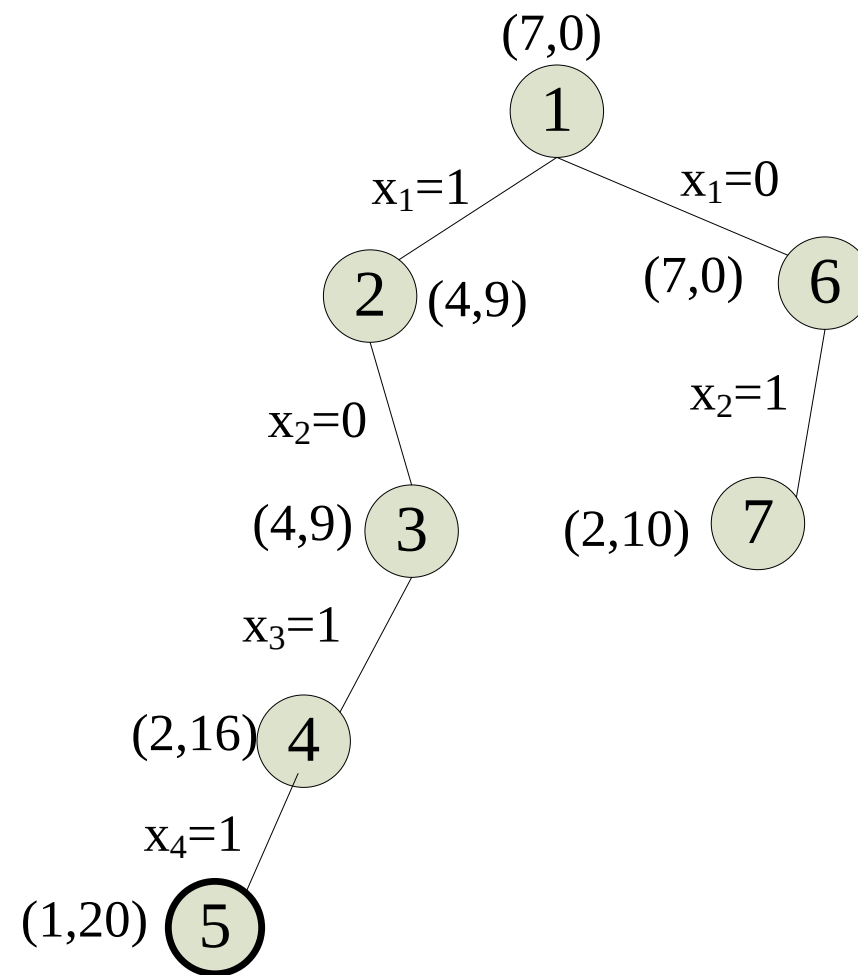
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 扩展节点6沿着左分支继续扩展，第二个物品的重量为5， $5 < 7$ ，满足约束条件



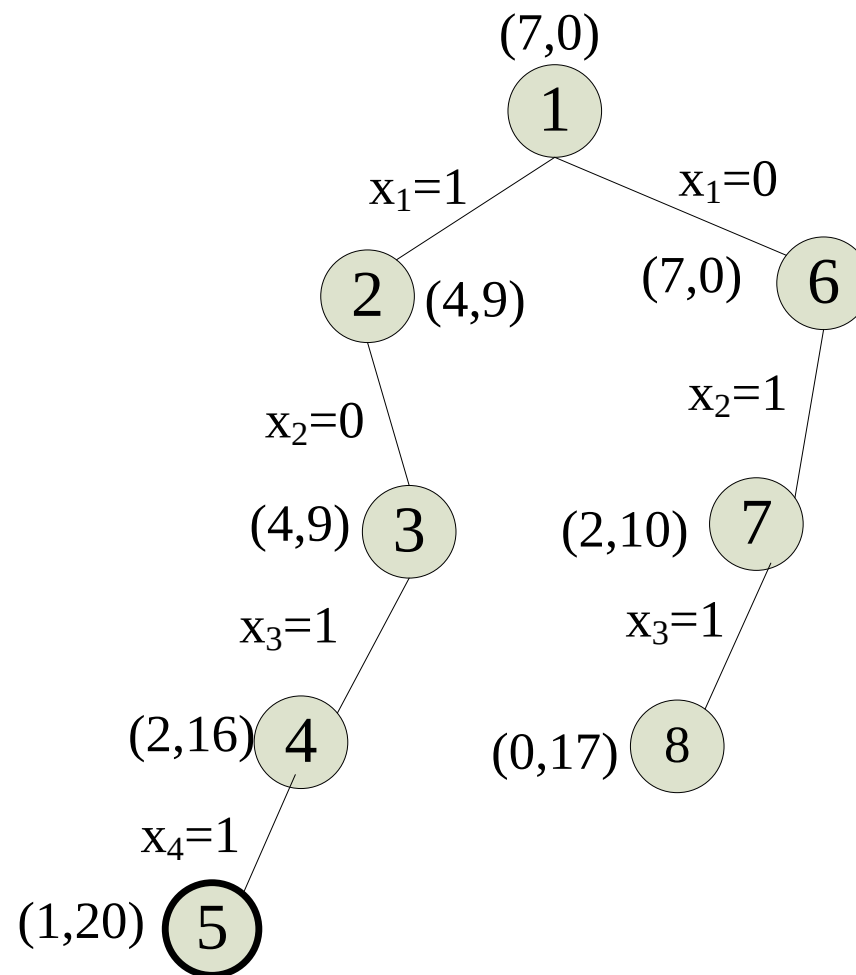
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 扩展节点7沿着左分支继续扩展, 判断约束条件, 当前背包剩余容量为2, 第3个物品的重量为2, 满足约束条件



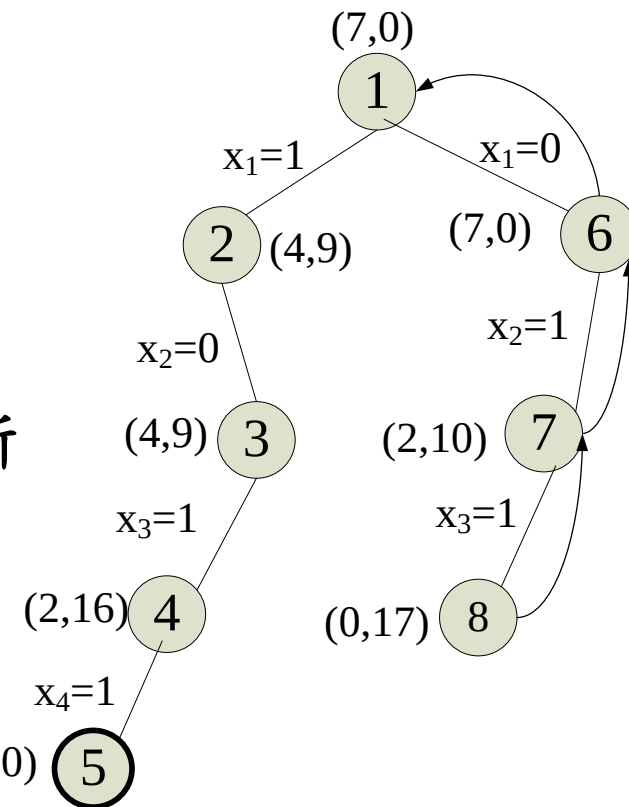
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

- 扩展节点8沿着左分支继续扩展, 当前背包剩余容量为0, 第4个物品的重量为1, $0 < 1$, 不满足约束条件
- 沿着扩展节点8的右分支进行扩展, 判断限界条件, $cp=17$, $rp=0$, $bestp=20$, $cp+rp < bestp$, 不满足限界条件
- 回溯到最近的活节点7, 扩展节点7沿着右分支继续扩展, 判断限界条件, 当前 $cp=10$, $rp=4$, $bestp=20$, $cp+rp < bestp$, 限界条件不满足
- 回溯到活节点6, 扩展节点6沿着右分支继续扩展, 当前 $cp=0$, $rp=11$, $bestp=20$, $cp+rp < bestp$, 限界条件不满足
- 回溯到活节点1, 节点1又成为扩展节点。节点1的分支全部搜索完毕, 节点1成为死节点, 搜索结束。



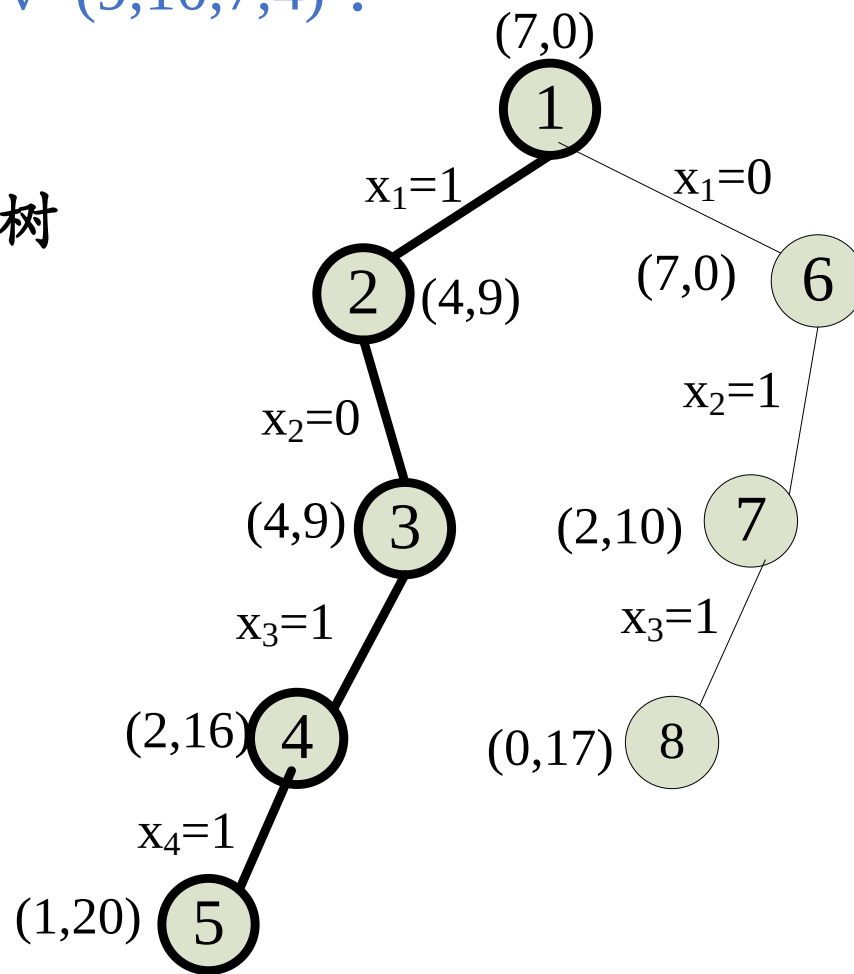
5.6 0-1背包问题



搜索解空间

$n=4$, $W=7$, $w=(3,5,2,1)$, $v=(9,10,7,4)$:

■ 示例0-1背包问题的搜索树



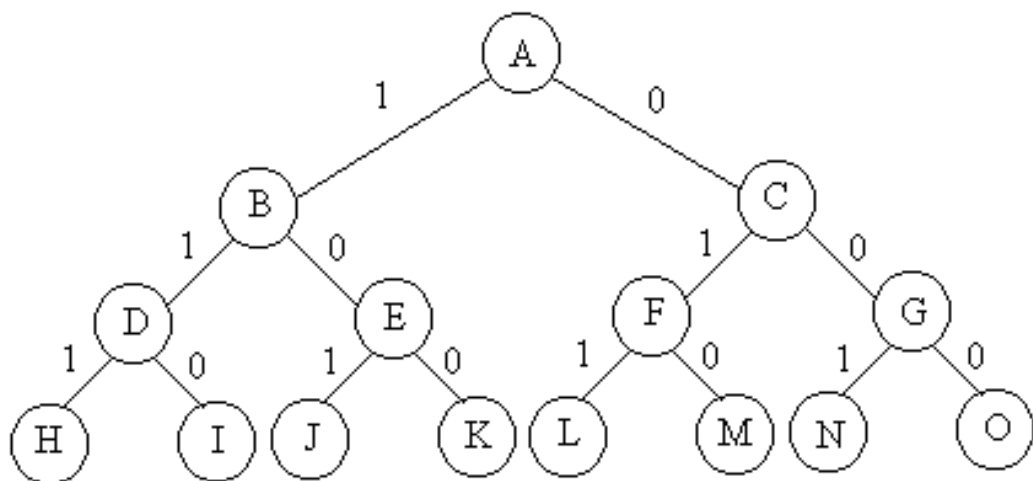
5.6 0-1背包问题



■ 解空间：子集树

■ 可行性约束函数： $\sum_{i=1}^n w_i x_i \leq c_1$

■ 上界函数：



```
template<class Typew, class Typep>
Typep Knap<Typew, Typep>::Bound(int i){// 计算上界rp
    Typew cleft = c - cw; // 剩余容量
    Typep b = cp;
    // 以物品单位重量价值递减序装入物品
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i];
        b += p[i];
        i++;
    }
    // 装满背包
    if (i <= n) b += p[i]/w[i] * cleft;
    return b;
}
```

■ 复杂性分析

- 判断约束函数需 $O(1)$ ，在最坏情况下有 $2^n - 1$ 个左孩子，约束函数耗时最坏为 $O(2^n)$ 。
- 计算上界限界函数需要 $O(n)$ 时间，在最坏情况下有 $2^n - 1$ 个右孩子需要计算上界，限界函数耗时最坏为 $O(n2^n)$ 。
- 0-1背包问题的回溯算法所需的计算时间为 $O(2^n) + O(n2^n) = O(n2^n)$ 。

0-1背包问题动态规划解法的复杂度： $O(n2^n)$ ，
受控跳跃点改进后的方法复杂度 $O(\min\{nc, 2^n\})$

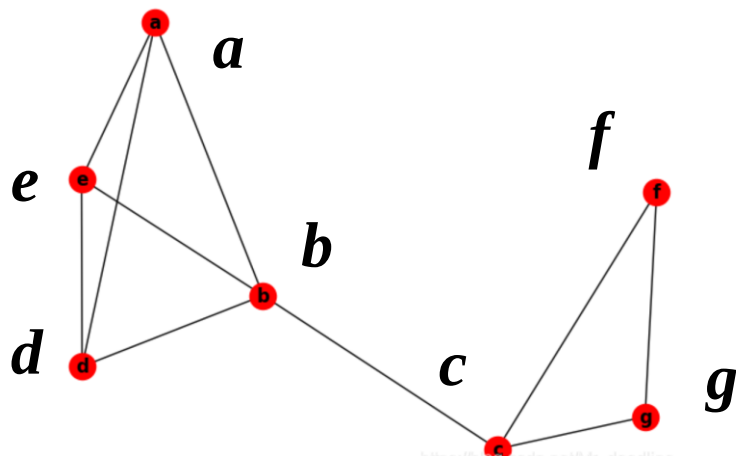
```
void Knap::Backtrack(int t)
{
    if(t>n)//到达叶子节点
    { for(int j=1;j<=n;j++)
        bestx[j]=x[j];
        bestp=cp;
        return;
    }
    if(cw+w[t]<=c) //搜索左子树
    { x[t]=1;
        cw+=w[t];
        cp+=p[t];
        Backtrack(t+1);
        cw-=w[t];
        cp-=p[t];
    }
    if(Bound(t+1)>bestp)//搜索右子树
    { x[t]=0;
        Backtrack(t+1); }
}
```

5.7 最大团问题



完全图，完全子图，最大完全子图

- **完全图**：任意两点都恰有一条边相连的图(任意两点都相邻)。
- **完全子图**：满足任意两点都恰有一条边相连的子图。
- **团**：不包含在更大的完全子图中。
- **最大完全子图**：所有完全子图中顶点数最大的团，即最大团，它的点集模最大。



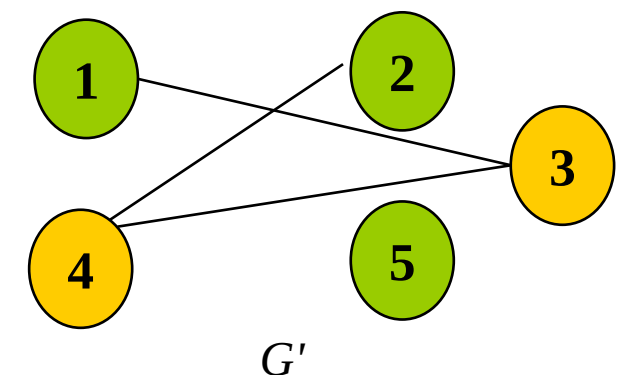
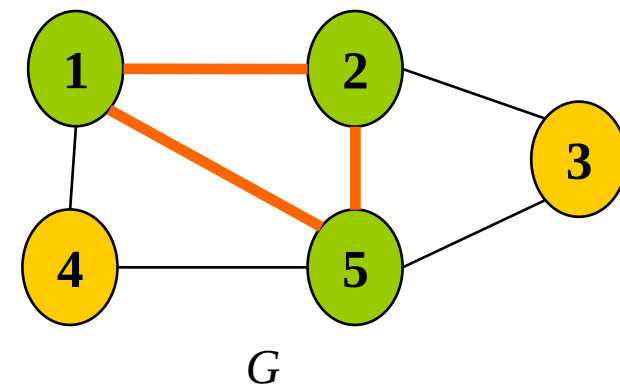
$\{a, b, d\}$, $\{a, e\}$, $\{c, f, g\}$ 等都是完全子图

最大完全子图为 $\{a, b, d, e\}$

5.7 最大团问题



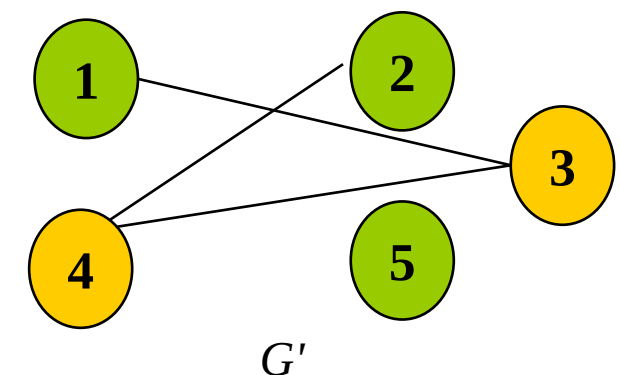
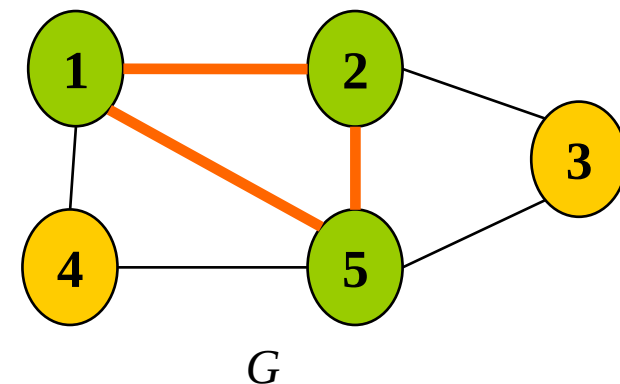
- 给定无向图 $G=(V,E)$ 。如果 $U\subseteq V$ ，且对任意 $u,v\in U$ 有 $(u,v)\in E$ ，则称 U 是 G 的**完全子图**。 G 的完全子图 U 是 G 的**团**当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。
- 如果 $U\subseteq V$ 且对任意 $u,v\in U$ 有 $(u,v)\notin E$ ，则称 U 是 G 的**空子图**。 G 的空子图 U 是 G 的**独立集**当且仅当 U 不包含在 G 的更大的空子图中。 G 的**最大独立集**是 G 中所含顶点数最多的独立集。
- 对于任一无向图 $G=(V, E)$ 其补图 $G'=(V', E')$ 定义为： $V'=V$ ，且 $(u, v)\in E'$ 当且仅当 $(u,v)\notin E$ 。
- U 是 G 的最大团当且仅当 U 是 G' 的最大独立集



5.7 最大团问题



- 给定无向图 $G=\{V, E\}$ ，其中 $V=\{1,2,3,4,5\}$ ， $E=\{(1,2), (1,4), (1,5), (2,3), (2,5), (3,5), (4,5)\}$ 。根据最大团定义，子集 $\{1,2\}$ 是图 G 的一个大小为2的完全子图，但不是一个团，因为它包含于 G 的更大的完全子图 $\{1,2,5\}$ 之中。 $\{1,2,5\}$ 是 G 的一个最大团。 $\{1,4,5\}$ 和 $\{2,3,5\}$ 也是 G 的最大团。
- 右侧图是无向图 G 的补图 G' 。根据最大独立集定义， $\{2,4\}$ 是 G 的一个空子图，同时也是 G 的一个最大独立集。
- 虽然 $\{1,2\}$ 也是 G' 的空子图，但它不是 G' 的独立集，因为它包含在 G' 的空子图 $\{1,2,5\}$ 中。 $\{1,2,5\}$ 是 G' 的最大独立集。 $\{1,4,5\}$ 和 $\{2,3,5\}$ 也是 G' 的最大独立集。



5.7 最大团问题



- 问题分析：最大团问题就是要求找出无向图 $G=(V, E)$ 的 n 个顶点集合 $\{1, 2, 3, \dots, n\}$ 的一个子集，这个子集中的任意两个顶点在无向图 G 中都有边相连，且包含顶点个数是最多的。
- 解空间：子集树
- 约束条件：顶点 i 到已选入的顶点集合中每一个顶点都有边相连。

判断新节点 t 是否与已有节点都相连

```
int Place (int t) {  
    int OK=1;  
    for (int j=1; j<t; j++)  
        // 顶点t与顶点j不相连  
        if (x[j] && a[t][j]==0){  
            OK=0;  
            break;  
        }  
    return OK;  
}
```

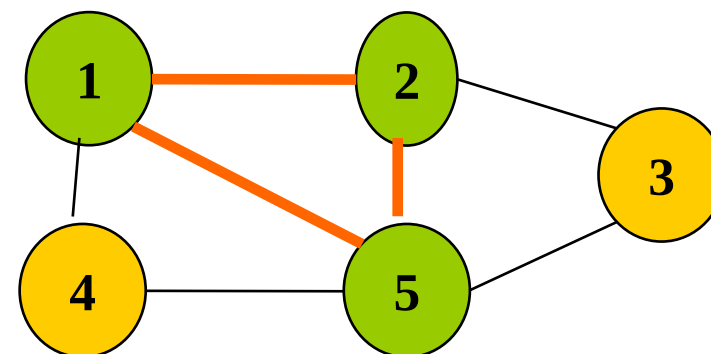
5.7 最大团问题



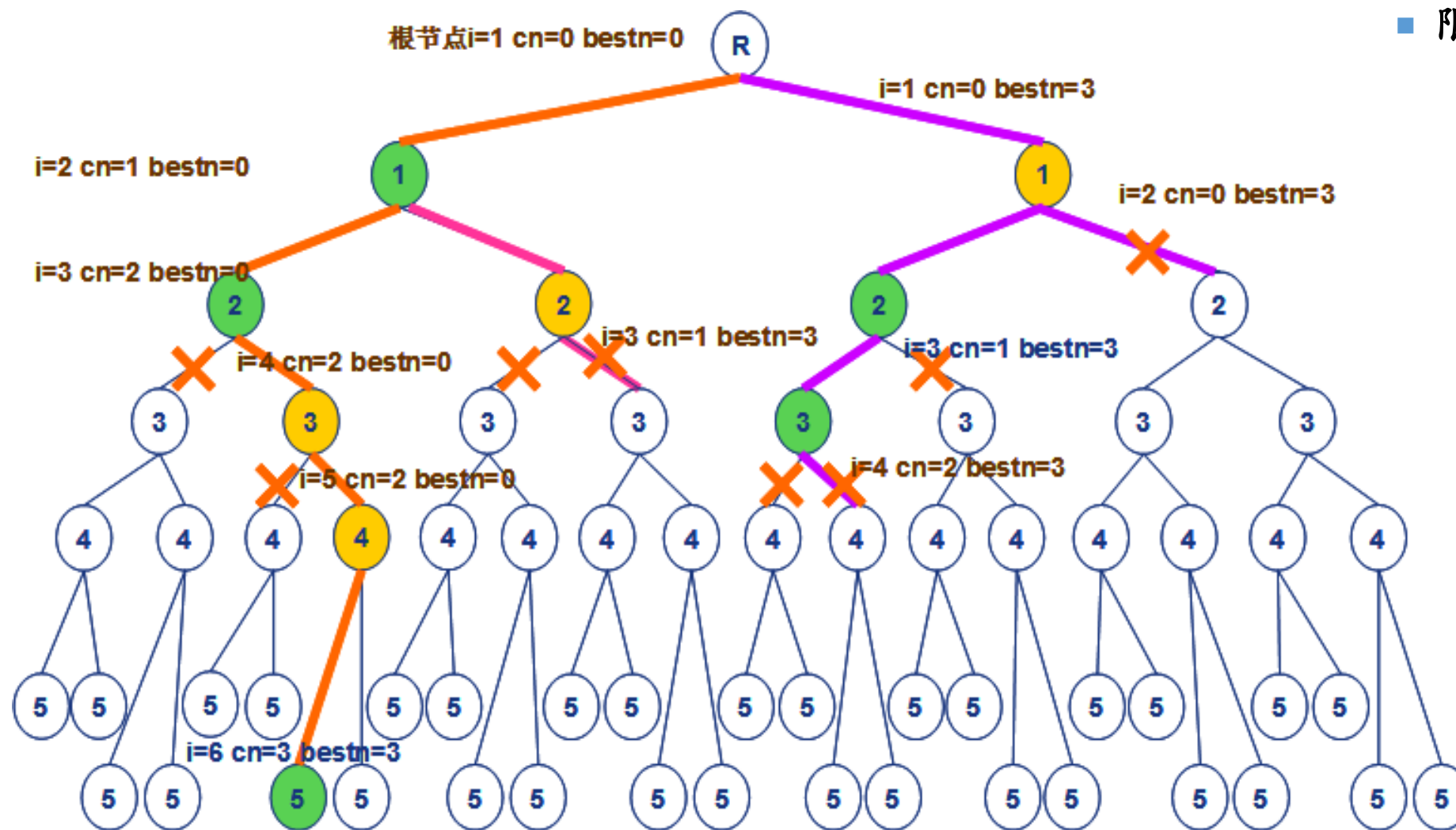
■ 限界条件

- $cn + rn > bestn$
- cn : 当前已包含在顶点集中的顶点个数
- rn : 剩余顶点个数($n-i$);
- $bestn$: 当前已找到的最优解包含的顶点个数

■ 搜索过程

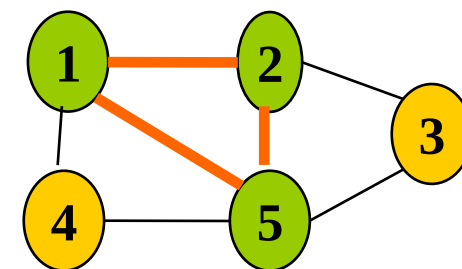


5.7 最大团问题



■ 限界条件

- $cn+rn > bestn$
- cn : 当前已包含在顶点集中的顶点个数
- rn : 剩余顶点个数($n-i$);
- $bestn$: 当前已找到的最优解包含的顶点个数

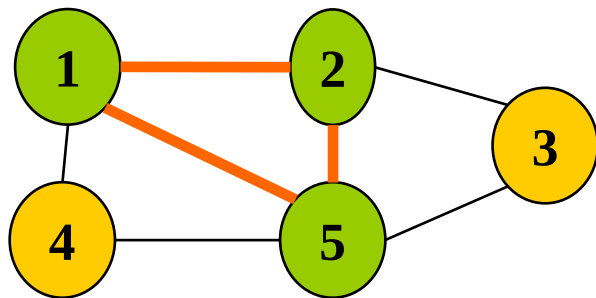


思考：为什么没有选出145,235作为最大团？

5.7 最大团问题



- 解空间：子集树
- 可行性约束函数：顶点*i*到已选入的顶点集中每一个顶点都有边相连。
- 上界函数：有足够多的可选择顶点使得算法有可能在右子树中找到更大的团。



复杂度分析

最大团问题的回溯算法backtrack所需的计算时间为 $O(2^n)$ 。

```
void Clique::Backtrack(int i){// 计算最大团
    if (i > n) {// 到达叶节点
        for (int j = 1; j <= n; j++) bestx[j] = x[j];
        bestn = cn; return;}
    // 检查顶点 i 与当前团的连接
    int OK = 1;
    for (int j = 1; j < i; j++)
        if (x[j] && a[i][j] == 0) {
            // i与j不相连
            OK = 0; break;}
    if (OK) {// 进入左子树
        x[i] = 1; cn++;
        Backtrack(i+1); x[i] = 0; cn--;}
    if (cn + n - i > bestn) {// 进入右子树
        x[i] = 0;
        Backtrack(i+1);}
}
```

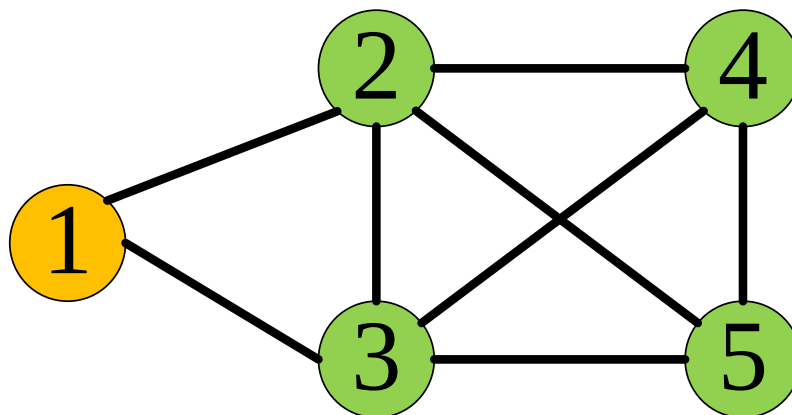
■ 算法分析

- 判断约束函数需 $O(n)$ ，在最坏情况下有 $2^n - 1$ 个左孩子，耗时最坏为 $O(n2^n)$ 。
- 判断限界函数需要 $O(1)$ 时间，在最坏情况下有 $2^n - 1$ 个右孩子节点需要判断限界函数，耗时最坏为 $O(2^n)$ 。
- 最大团问题的回溯算法所需的计算时间为 $O(2^n) + O(n2^n) = O(n2^n)$ 。

5.7 最大团问题



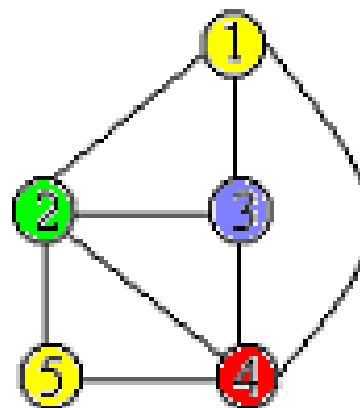
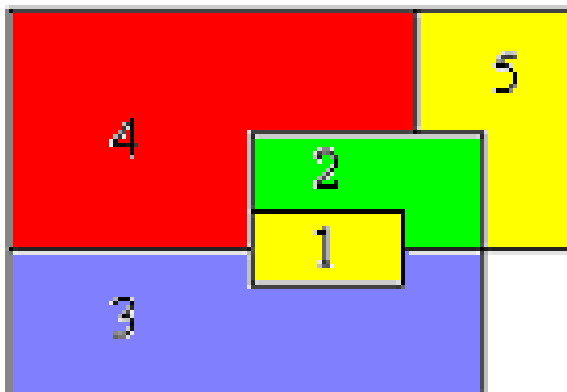
■ 搜索过程?



5.8 图的m着色问题



- 给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。是否有一种着色法使 G 中每条边的2个顶点着不同颜色。这个问题是图的 m 可着色判定问题。
- 若一个图最少需要 m 种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的 m 可着色优化问题。



5.8 图的 m 着色问题



- 图着色问题描述为：给定无向连通图 $G=(V, E)$ 和正整数 m ，求最小的整数 m ，使得用 m 种颜色对 G 中的顶点着色，使得任意两个相邻顶点着色不同。
- 整数 m 为该图的着色数。求一个图的色数 m 的问题称为图的 m 可着色最优化问题。

5.8 图的m着色问题



- 设无向图 $G=(V, E)$ 采用如下邻接矩阵表示:

$$a[i][j] = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{others} \end{cases}$$

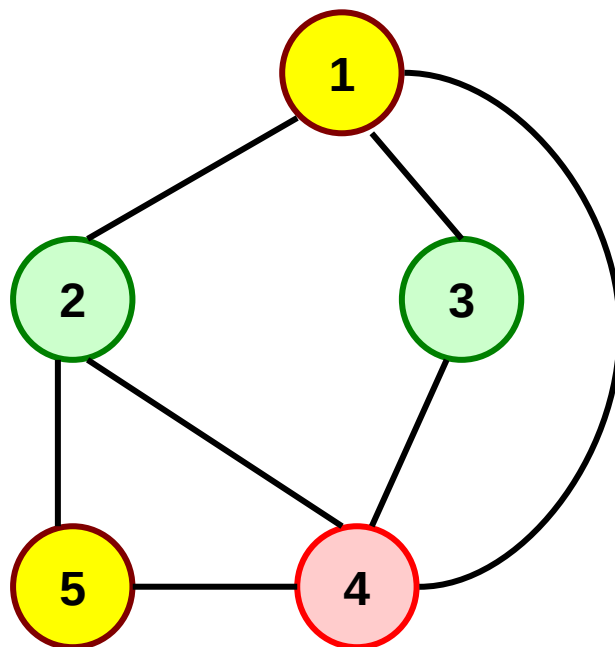
- 由于用 m 种颜色为无向图 $G=(V, E)$ 着色, 其中, V 的顶点个数为 n , 可以用一个 n 元组 $(x_1, x_2, \dots, x_n)$ 来描述图的一种可能着色, 其中, $x_i \in \{1, 2, \dots, m\} (1 \leq i \leq n)$ 表示赋予顶点 i 的颜色, 这是显式约束。 $x_i=0$ 表示没有可用的颜色。因此解空间大小为 m^n 。
- 约束函数从隐式约束产生: 对所有 i 和 j ($1 \leq i, j \leq n, i \neq j$), 若 $a[i][j]=1$, 则 $x_i \neq x_j$ 。

5.8 图的m着色问题



举例

- 例如，5元组(1, 2, 2, 3, 1)表示对具有5个顶点的无向图的一种着色，顶点1着颜色1，顶点2着颜色2，顶点3着颜色2，顶点4着颜色3，顶点5着颜色1。
- 如果在n元组中，所有相邻顶点都不着相同颜色，就称此n元组为可行解，否则为无效解。



5.8 图的m着色问题



回溯法求解

- 回溯法求解图着色问题，首先把所有顶点的颜色初始化为0，然后依次为每个顶点着色。
- 在图着色问题的解空间树中：
 - 如果从根节点到当前节点对应一个部分解，也就是所有的颜色指派都没有冲突，则在当前节点处选择第一棵子树继续搜索，也就是为下一个顶点着颜色1，
 - 否则，对当前子树的兄弟子树继续搜索，也就是为当前顶点着下一个颜色，
 - 如果所有m种颜色都已尝试过并且都发生冲突，则回溯到当前节点的父节点处，上一个顶点的颜色被改变，依此类推。

5.8 图的m着色问题



- 解向量: $(x_1, x_2, \dots, x_n)$ 表示顶点 i 所着颜色 $x[i]$
- 可行性约束函数: 顶点 i 与已着色的相邻顶点颜色不重复。

复杂度分析

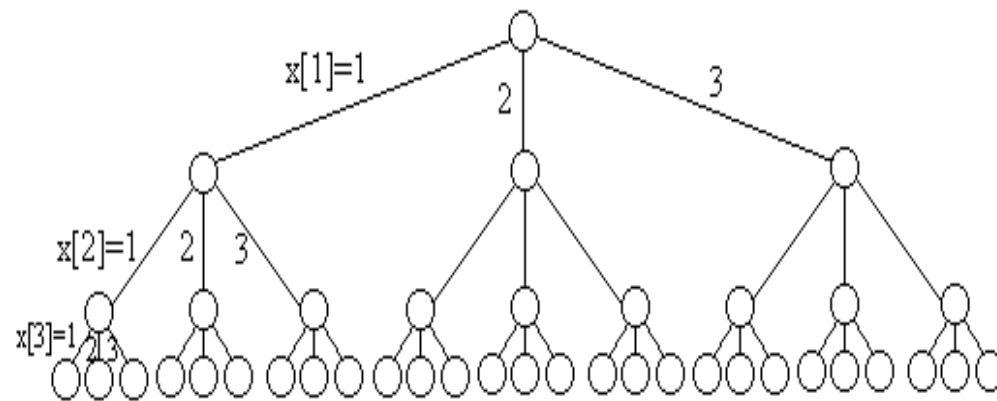
图 m 可着色问题的解空间树中内节点个数是 $\sum_{i=0}^{n-1} m^i$

对于每一个内节点, 在最坏情况下, 用 ok 检查当前扩展节点的每一个儿子所相应的颜色可用性需耗时 $O(mn)$ 。因此, 回溯法总的的时间耗费是

$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1) / (m - 1) = O(nm^n)$$

```
void Color::Backtrack(int t){  
    if (t>n) {  
        sum++;  
        for (int i=1; i<=n; i++)  
            cout << x[i] << ' ';  
        cout << endl; }  
    else  
        for (int i=1; i<=m; i++) {  
            x[t]=i;  
            if (Ok(t)) Backtrack(t+1);  
        }  
}
```

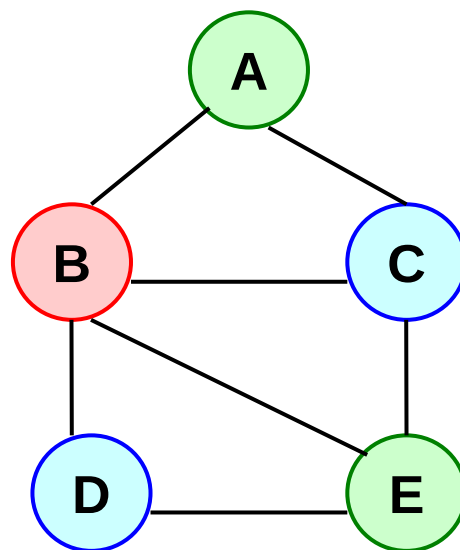
```
bool Color::Ok(int k){// 检查颜色可用性  
    for (int j=1; j<=n; j++)  
        if ((a[k][j]==1)&&(x[j]==x[k])) return false;  
    return true;  
}
```



5.8 图的m着色问题



■ 搜索过程?



5.9 旅行售货员问题



■ 问题描述

- 设有 n 个城市组成的交通图，一个售货员从驻地城市出发，到其它城市各一次去推销货物，最后回到驻地城市。假定任意两个城市 i, j 之间的距离 d_{ij} ($d_{ij}=d_{ji}$)是已知的，问应该怎样选择一条最短（或总旅费最小）的路线？

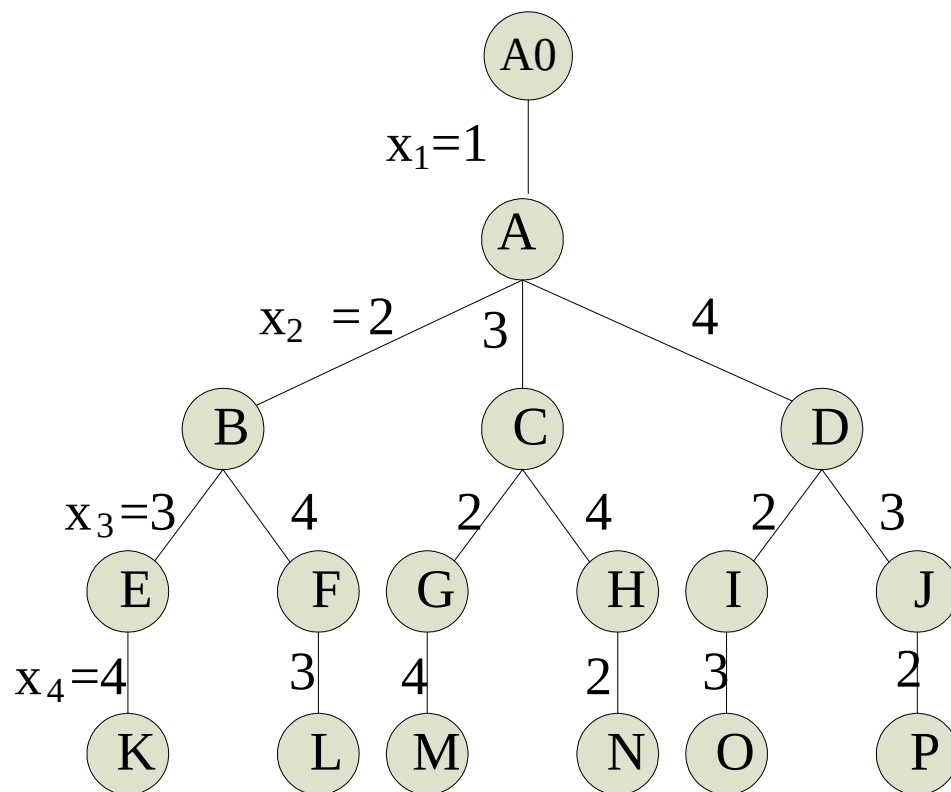
■ 定义问题的解空间

- 解的形式 $(x_1, x_2, \dots, x_n)$
- 令 n 个城市组成的集合为 $S=\{1, 2, \dots, n\}$ ，则 $x_1=1$ ， $x_i \in S - \{x_1, x_2, \dots, x_{i-1}\}$ ， $i=2, \dots, n$ 。

5.9 旅行售货员问题



解空间排列树



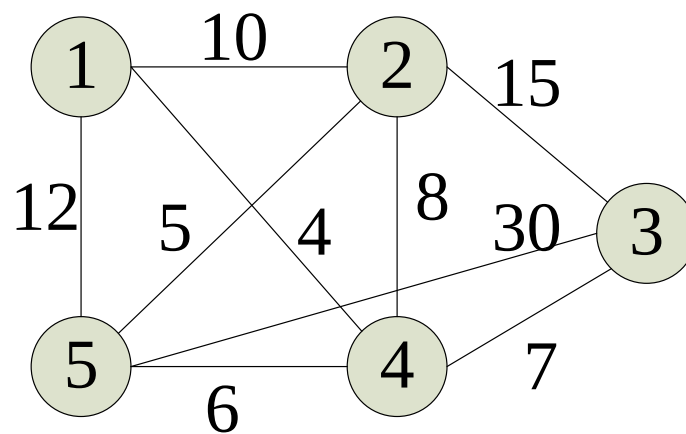
$n=4$ 的旅行售货员问题的解空间树

5.9 旅行售货员问题



搜索解空间

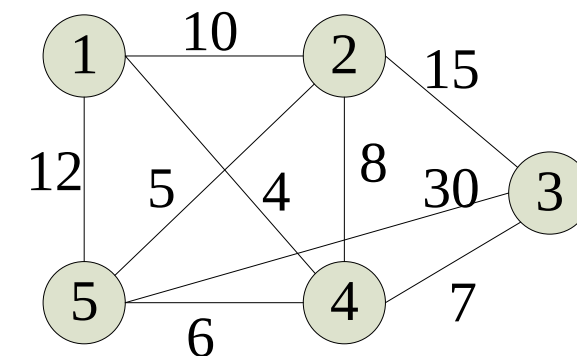
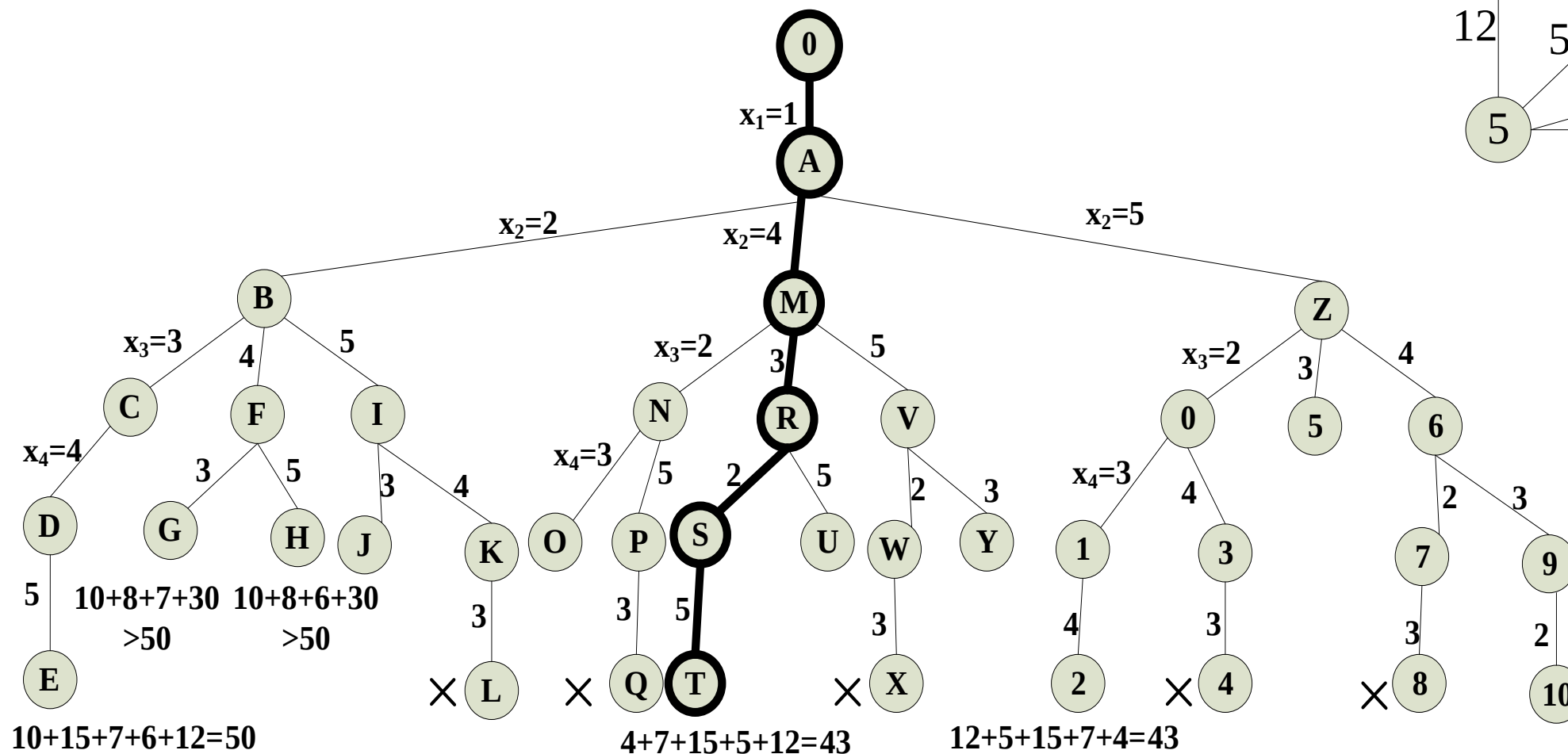
- 设置约束条件;
 - 用二维数组 $a[i][j]$ 存储无向带权图的邻接矩阵, 如果 $a[i][j] \neq \infty$ 表示城市 i 和城市 j 有边相连, 能走通。
- 设置限界条件
 - $cc < bestc$, cc 的初始值为0, $bestc$ 的初始值为 $+\infty$ 。
 - cc : 当前已走过的城市所用的路径长度
 - $bestc$: 表示当前找到的最短路径的路径长度
- 搜索过程: 以右图为例说明



5.9 旅行售货员问题



搜索树



5.9 旅行售货员问题



```
template<class Type>
void Traveling<Type>::Backtrack(int i){
    if (i == n) {
        if (a[x[n-1]][x[n]] != NoEdge && a[x[n]][1] != NoEdge &&
            (cc + a[x[n-1]][x[n]] + a[x[n]][1] < bestc || bestc == NoEdge)) {
            for (int j = 1; j <= n; j++) bestx[j] = x[j];
            bestc = cc + a[x[n-1]][x[n]] + a[x[n]][1]; }
    }
    else {
        for (int j = i; j <= n; j++)
            // 是否可进入x[j]子树?
            if (a[x[i-1]][x[j]] != NoEdge &&
                (cc + a[x[i-1]][x[j]] < bestc || bestc == NoEdge)) {
                // 搜索子树
                Swap(x[i], x[j]);
                cc += a[x[i-1]][x[i]];
                Backtrack(i+1);
                cc -= a[x[i-1]][x[i]];
                Swap(x[i], x[j]); }
    }
}
```

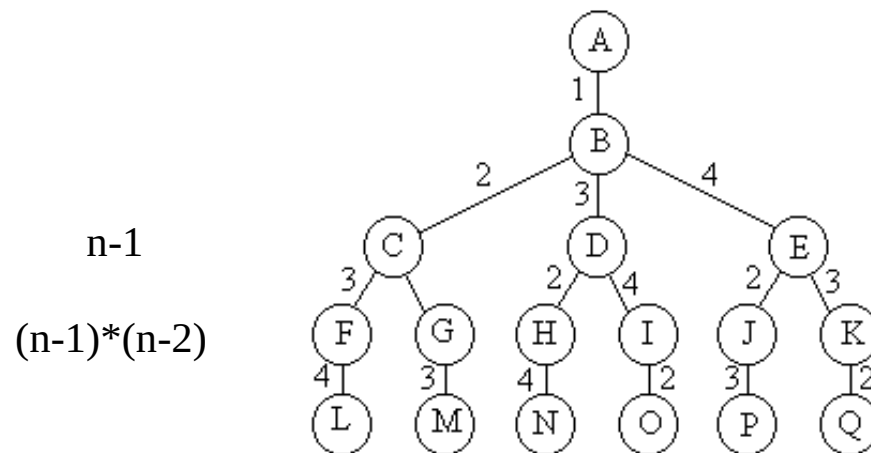
■ 解空间：排列树

5.9 旅行售货员问题



■ 算法分析

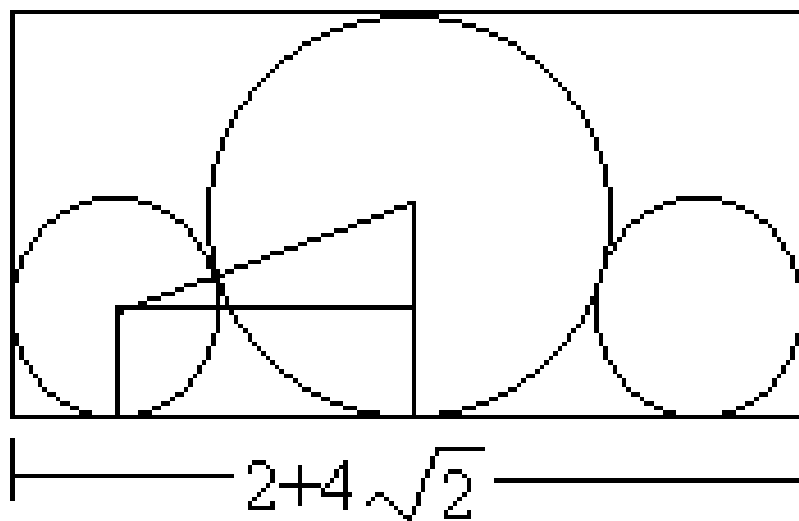
- 判断限界函数需要 $O(1)$ 时间，在最坏情况下有 $1+(n-1)+(n-1)(n-2)+\dots+(n-1)(n-2)\dots 2*1 \leq n(n-1)!$ 个节点需要判断限界函数，故耗时 $O(n!)$;
- 在叶子节点处记录当前最优解需要耗时 $O(n)$ ，在最坏情况下会搜索到每一个叶子节点，叶子节点有 $(n-1)!$ 个，故耗时为 $O(n!)$ 。
- 旅行售货员问题的回溯算法所需的计算时间为 $O(n!)+O(n!)=O(n!)$ 。



5.10 圆排列问题



- 给定 n 个大小不等的圆 $c_1, c_2, \dots, c_n$ ，现要将这 n 个圆排进一个矩形框中，且要求各圆与矩形框的底边相切。圆排列问题要求从 n 个圆的所有排列中找出有最小长度的圆排列。例如，当 $n=3$ ，且所给的3个圆的半径分别为1, 1, 2时，这3个圆的最小长度的圆排列如图所示。其最小长度为 $2+4\sqrt{2}$



5.10 圆排列问题



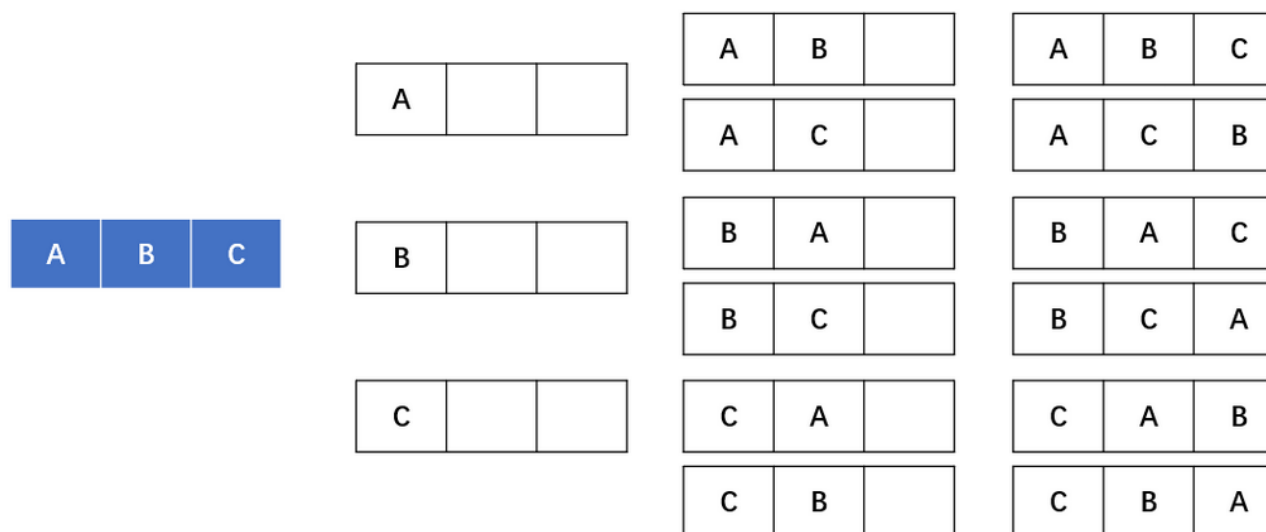
- 圆排列问题的解空间是一棵排列树，我们用回溯法在整个排列数中搜索最优解。
- 首先，我们可以把这个问题分解为以下几步：
 - 找出所有圆的排列
 - 计算出每一个排列的长度
 - 选择最小的长度

5.10 圆排列问题



(一) 全排列

- 给你一组半径，每个代表一个圆，找出所有的排列方式。这其实就是全排列问题。
- 我们用递归的方法找出全部的排列。
- 比如：给你ABC



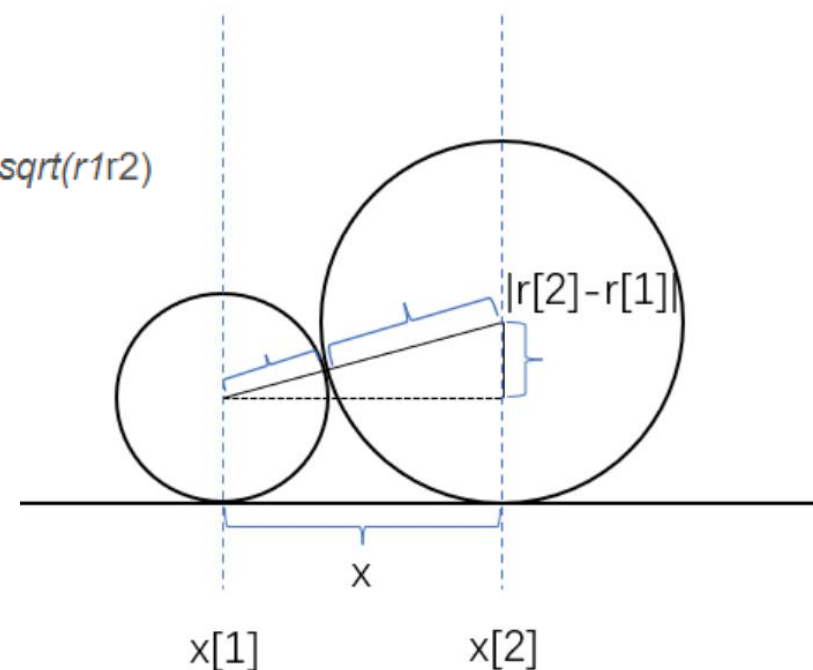
(二) 计算一种排列的长度

- 找出了所有的排列，那么只要我们能够计算出一种排列的长度
- 对所有的排列 比较一个最小的即可得到答案
- 那么假设前一个圆的圆心横坐标为 x_1 ，后一个是 x_2 ，半径是 r_1 和 r_2

- 那么根据勾股定理即可计算出 x ：推导出 x ，

$$x^2 = \text{sqrt}((r_1 + r_2)^2 - (r_1 - r_2)^2) \text{推导出 } x = 2\text{sqrt}(r_1 r_2)$$
$$x_2 = x_1 + x$$

- 那么有了这个公式，我们假定第一个圆形的圆心横坐标为0，知道了第一个就能算出第二个，以此类推所有的圆心横坐标就都算出来了。



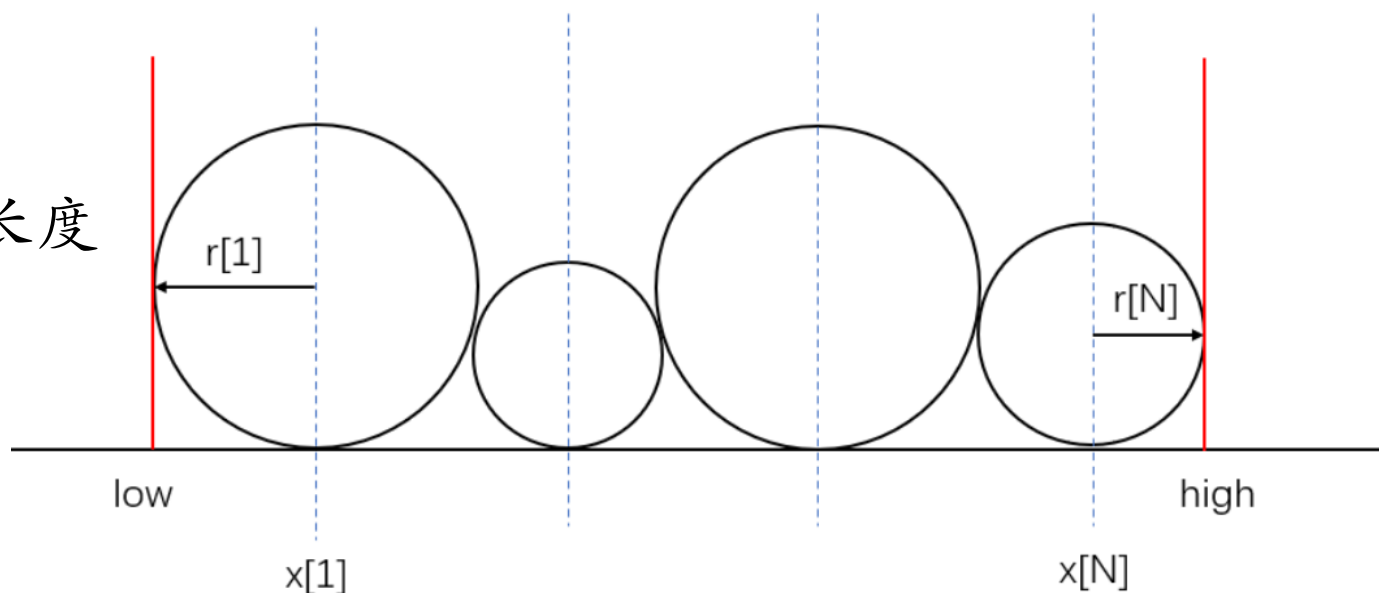
5.10 圆排列问题



(二) 计算一种排列的长度

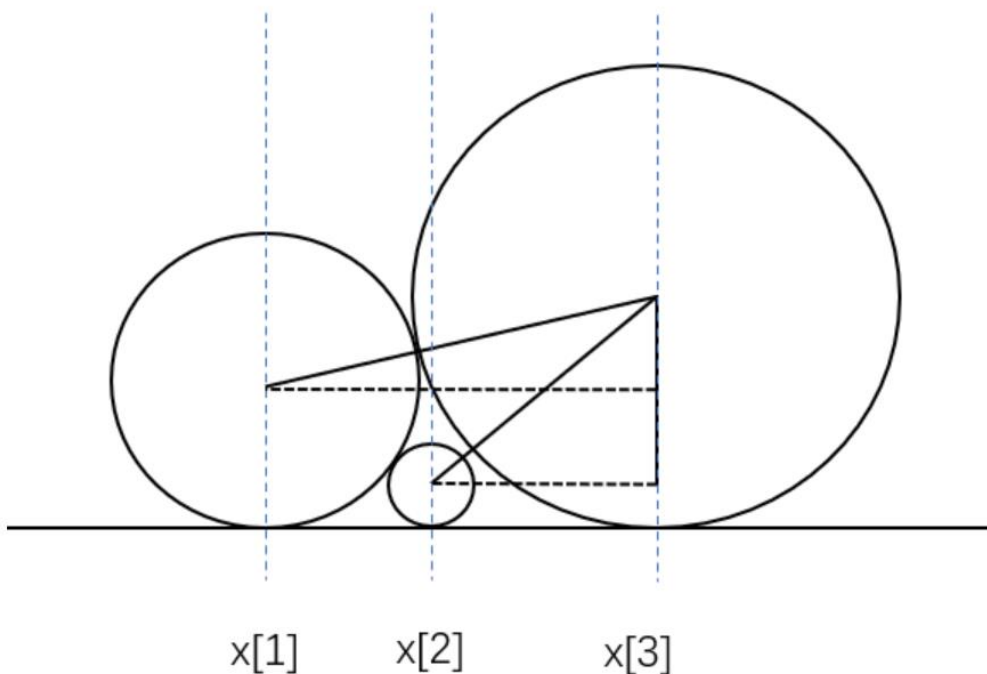
- 有了所有的横坐标，还有该排列所有圆的半径
- 排列的长度：
 - 那么第一个圆的横坐标加上第一个圆的半径就是该排列的最左边low
 - 最后一个圆的横坐标加上最后一个圆的半径就是该排列的最右边high

high-low即可得到长度



(三) 完善算法

- 前面的想法其实还存在一些问题。
- 计算横坐标 x 的时候，我们使用的公式有一个前提，就是这个圆必须和前一个圆相切，那么实际情况中是相切的嘛？

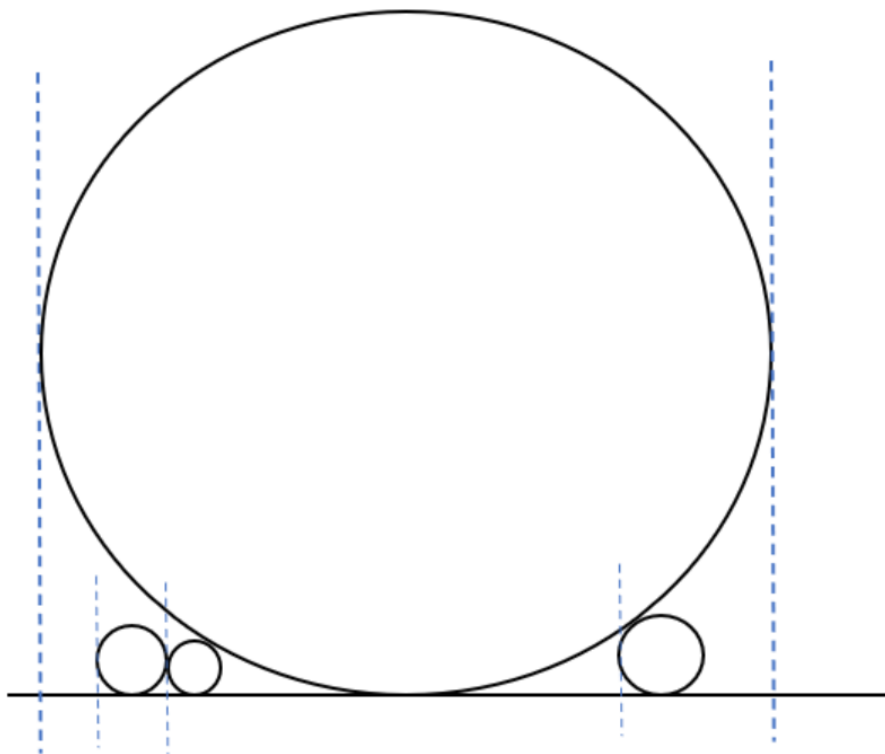


如图，是不一定的。当前圆并不一定与前一个圆相切，有可能和前一个相切，也有可能和前面任意一个圆相切。所以，我们其实需要和前面所有的圆都进行一次计算。

如图，这个圆和前面相邻的那个并不相切，所以你用我们说的那个公式计算的话，会计算出来偏小的结果。所以我们只需要把当前圆与前面所有的圆依次计算，保留最大的值，就是正确的值。

(三) 完善算法

- 我们计算左右边界的方法是对的吗?
- 左边界并不一定是第一个圆的左边



如图，左边界并不是第一个圆的左边，那么怎么算呢

其实我们有了每个圆的圆心横坐标和半径了，我们只要把每个圆的左边界算出来，取最小的就是整个排列的左边界了

同理，把每个圆的右边界算出来，取最大的，就是整个排列的右边界了

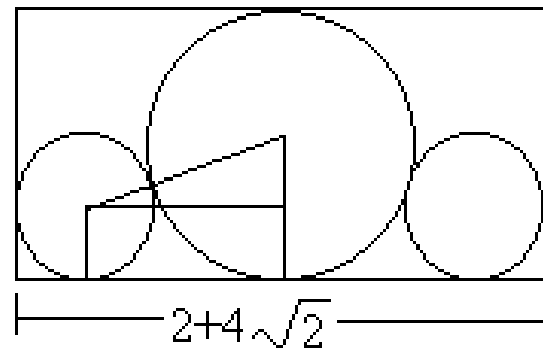
5.10 圆排列问题



```
void Circle::Backtrack(int t){
    if (t>n) Compute();
    else
        for (int j = t; j <= n; j++) {
            Swap(r[t], r[j]);
            float centerx=Center(t);
            if (centerx+r[t]+r[1]<min) { // 下界约束
                x[t]=centerx;
                Backtrack(t+1);
            }
            Swap(r[t], r[j]);
        }
}
```

```
float Circle::Center(int t)
{// 计算当前所选择圆的圆心横坐标
    float temp=0;
    for (int j=1;j<t;j++) {
        float valuex=x[j]+2.0*sqrt(r[t]*r[j]);
        if (valuex>temp) temp=valuex;
    }
    return temp;
}

void Circle::Compute(void)
{// 计算当前圆排列的长度
    float low=0,
        high=0;
    for (int i=1;i<=n;i++) {
        if (x[i]-r[i]<low) low=x[i]-r[i];
        if (x[i]+r[i]>high) high=x[i]+r[i];
    }
    if (high-low<min) min=high-low;
}
```



5.10 圆排列问题



上述算法尚有许多改进的余地。例如，像 $1, 2, \dots, n-1, n$ 和 $n, n-1, \dots, 2, 1$ 这种互为镜像的排列具有相同的圆排列长度，只计算一个就够了，可减少约一半的计算量。另一方面，如果所给的 n 个圆中有 k 个圆有相同的半径，则这 k 个圆产生的 $k!$ 个完全相同的圆排列，只计算一个就够了。

5.12 回溯法效率分析



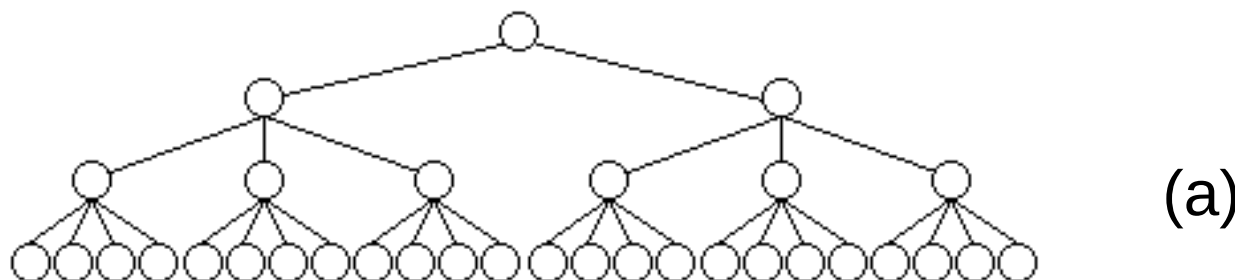
- 通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：
 - (1)产生 $x[k]$ 的时间；
 - (2)满足显约束的 $x[k]$ 值的个数；
 - (3)计算约束函数constraint的时间；
 - (4)计算上界函数bound的时间；
 - (5)满足约束函数和上界函数约束的所有 $x[k]$ 的个数。
- 好的约束函数能显著地减少所生成的节点数。但这样的约束函数往往计算量较大。
因此，在选择约束函数时通常存在生成节点数与约束函数计算量之间的折衷。

5.12 回溯法效率分析

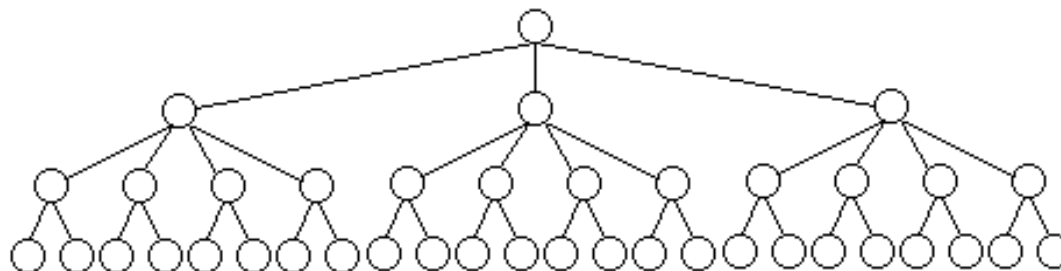


重排原理

对于许多问题而言，在搜索试探时选取 $x[i]$ 的值顺序是任意的。在其它条件相当的前提下，让可取值最少的 $x[i]$ 优先。从图中关于同一问题的2棵不同解空间树，可以体会到这种策略的潜力。



(a)



(b)

图(a)中，从第1层剪去1棵子树，则从所有应当考虑的3元组中一次消去12个3元组。对于图(b)，虽然同样从第1层剪去1棵子树，却只从应当考虑的3元组中消去8个3元组。前者的效果比后者好。